Implementation of FastSpeech 2-Based Text-to-Speech Synthesis for English and Telugu

Project Report

Submitted by

Savio Shaju

Department of Computer Science and Engineering

Saintgits College of Engineering

GitHub Repository

https://github.com/savioshaju/TTS.git

Project Outputs and Model Checkpoints

https://drive.google.com/drive/folders/1MYS5zcsyehxDN5n4XaKj5jCidpUZ-efw?usp=sharing

Contents

1 Introduction **1**

2 Project Setup and Repository Architecture **1**
2.1 Base Repository Selection and Environment Setup . . . 1
2.2 Code Remediation and Upstream Bug Fixes . . . 2

3 Dataset Preparation and Corpus Analysis **2**
3.1 Corpus Selection and Assignment Duration Requirements . . . 2
3.2 Audio Standardization, Downsampling, and Quality Filtering . . . 2
3.3 Stratified Train-Validation Splitting (5 + 1 Hour Allocation) . . . 3

4 Text Preprocessing and Phonetic Representation **4**
4.1 English Text Processing Pipeline . . . 4
4.2 Telugu Text Processing Pipeline . . . 4

5 Forced Alignment and Duration Extraction **5**
5.1 English Alignment via Montreal Forced Aligner (MFA) . . . 5
5.2 Telugu Alignment via Meta MMS Forced Alignment . . . 5

6 Acoustic Feature Extraction and Normalization **5**
6.1 Mel-Spectrogram Generation . . . 5
6.2 Pitch (F_0) Extraction and Preprocessing . . . 6
6.3 Energy Extraction . . . 6
6.4 Dataset-Wide Feature Normalization . . . 6

7 Model Architecture and Implementation Details **7**
7.1 Overview of the FastSpeech 2 Pipeline . . . 7
7.2 Encoder and Decoder . . . 7
7.3 Variance Adaptor . . . 7
7.4 Post-Net and Multi-Task Loss Formulation . . . 8
7.5 HiFi-GAN Neural Vocoder and the Resampling Bridge . . . 8

8 Training Configuration, Environment, and Loss Dynamics **8**
8.1 Hardware and Hyperparameters . . . 8
8.2 Two-Phase Training Trajectory and Convergence . . . 9
8.3 Checkpoint Selection Audit and Overfitting Analysis . . . 10

9 Speech Synthesis Evaluation and Variance Adaptor Analysis **10**
9.1 Inference Execution and Checkpoint Loading . . . 10
9.2 Quantitative Benchmark Performance Across Prescribed Sentences . . . 10
9.3 Prescribed English Benchmark Sentences and Variance Profiles . . . 11
9.4 Prescribed Telugu Benchmark Sentences and Variance Profiles . . . 14

10 Cross-Linguistic Comparative Analysis, Failure Modes, and Technical Recommendations **17**
10.1 Linguistic Comparison: Indian English vs. Telugu . . . 17
10.2 Recommendations Prior to Local Fine-Tuning . . . 17

11 Reproducibility Details and Checkpoint Distribution **17**
11.1 Execution Instructions . . . 17
11.2 Model Weights and Checkpoint Distribution . . . 17

12 Conclusion **18**

1 Introduction

Text-to-Speech (TTS) synthesis has undergone a major paradigm shift from concatenative and statistical parametric models to deep neural network acoustic models. Early neural systems such as Tacotron 2 achieved state-of-the-art naturalness but suffered from three inherent drawbacks of autoregressive sequence-to-sequence generation: slow inference speed due to step-by-step frame decoding, word skipping or repeating caused by imperfect cross-attention alignment, and limited prosodic controllability. To overcome these limitations, non-autoregressive architectures were introduced, led by FastSpeech and FastSpeech 2. FastSpeech 2 eliminates sequential generation by utilizing Feed-Forward Transformer (FFT) blocks to synthesize mel-spectrograms in parallel. Furthermore, it directly incorporates explicit variance predictors for phoneme duration, fundamental frequency (F_0), and signal energy, training them directly against ground-truth acoustic targets rather than relying on distilled teacher knowledge.

The objective of this project is to implement, adapt, and comprehensively evaluate the FastSpeech 2 architecture separately for two distinct languages: Indian English and Telugu. While English represents a stress-timed Germanic language with deep orthographic irregularities and a wide variety of regional accents, Telugu is a syllable-timed Dravidian language characterized by phonetic agglutination, complex consonant clusters (*samyuktakshara*), and an akshara-based writing system. Implementing FastSpeech 2 across both linguistic domains requires designing distinct text preprocessing pipelines, forced alignment strategies, and acoustic target extractors, while maintaining a unified, robust acoustic modeling and neural vocoding framework.

This report documents the end-to-end engineering lifecycle of the project. It describes the initial repository setup and defect remediation, corpus preparation from the Svarah and IndicTTS datasets, forced alignment workflows, acoustic feature extraction, neural network training dynamics up to 100,000 steps, inference execution, and detailed phonetic evaluation of the synthesized test sentences.

2 Project Setup and Repository Architecture

2.1 Base Repository Selection and Environment Setup

The implementation began with the reference open-source PyTorch implementation of FastSpeech 2 (`FastSpeech2-master`), which originally targeted single-speaker English synthesis on the LJSpeech corpus. The project was configured inside a dedicated Python 3.11 virtual environment (`.venv`) with core scientific and machine learning dependencies including PyTorch 2.5, TorchAudio, NumPy, SciPy, Librosa, PyWorld (for Harvest pitch extraction), and Matplotlib.

To accommodate both languages independently while avoiding configuration collisions, the project repository was restructured into decoupled, language-specific directory trees:

- `English/`: Houses the configuration file (`config.yaml`), training pipeline (`train/`), inference scripts (`inference/`), and model weights directory (`checkpoints/`).
- `Telugu/`: Houses the Telugu-specific configuration, phoneme cleaners, MMS alignment scripts, training loops, and checkpoints.
- `Results/`: Organised directory storing generated audio WAV files, evaluation metrics, and variance adaptor plots.
- `evaluation/`: Complete standalone evaluation benchmark suite containing audio waveforms, multi-panel plots, and quantitative tensor metrics.
- `preprocessed_data/`: Stores extracted mel-spectrograms, pitch contours, energy contours, and token alignments.

2.2 Code Remediation and Upstream Bug Fixes

During the initial code audit of the upstream repository, three critical implementation defects were identified and resolved:

1. **Variance Adaptor Control Leak (`model/modules.py`):** In the upstream implementation, the user-specified pitch control scale (`p_control`) was mistakenly passed to the energy predictor's embedding projection (`get_energy_embedding`) instead of `e_control`. Consequently, energy scaling was completely ignored during inference, while pitch scaling inadvertently modulated both pitch and energy simultaneously. This was corrected by separating the scaling arguments.

2. **Multi-GPU Device Hardcoding (`utils/tools.py`):** Hardcoded references to `torch.device("cuda")` caused device mismatch runtime errors under `torch.nn.DataParallel` when training on dual-GPU environments. All device allocations were modified to dynamically inherit the device ID of the current batch tensor.

3. **G2P Fallback Behavior (`text/__init__.py`):** The default text cleaner fell back to character-level tokenization if raw text lacked enclosing curly braces `{...}`. Because the acoustic model was trained strictly on phoneme sequences, passing raw text resulted in character IDs being mapped to phoneme embedding indices, producing unintelligible audio. A pre-validation check was added to ensure input sequences are properly converted before embedding lookup.

3 Dataset Preparation and Corpus Analysis

3.1 Corpus Selection and Assignment Duration Requirements

The assignment specification explicitly mandates using **approximately 5 hours of audio for training and 1 hour of audio for validation** for each language, with complete separation between splits and detailed documentation of the subsampling strategy. Two distinct speech corpora were curated to fulfill these requirements:

- **English (AI4Bharat Svarah Corpus):** Svarah is an authentic crowdsourced speech repository designed to capture the wide regional and demographic diversity of Indian English. A high-fidelity subset of 4,202 utterances was sampled from 104 distinct speakers (54 female, 50 male) representing 19 native language substrates (including Hindi, Telugu, Tamil, Kannada, Marathi, Bengali, Odia, Gujarati, Malayalam, and Urdu). After forced alignment and acoustic validation, **4,165 verified utterances** totaling **6.07 hours** were retained, precisely targeting the 5-hour training and 1-hour validation requirement.

- **Telugu (IndicTTS Corpus):** The Telugu dataset was acquired from the IndicTTS initiative (developed by IIT Madras and MeitY). The raw female corpus contains over 29 hours of studio recordings at 48 kHz, while the male corpus was recorded at 16 kHz. To prevent gender imbalance while strictly satisfying the ≈ 5-hour training and 1-hour validation constraint, a gender-balanced subset of 2,695 utterances (5.00 hours) was constructed (1,348 female utterances, 1,347 male utterances). After removing two corrupted files during forced alignment validation, **2,693 verified utterances** totaling **5.00 hours** (4.99 hours) were retained.

3.2 Audio Standardization, Downsampling, and Quality Filtering

Because the raw source corpora possessed divergent acoustic configurations (16 kHz crowdsourced recordings for Svarah versus 48 kHz studio recordings for IndicTTS female audio), a standardized acoustic ingestion pipeline was implemented:

- **High-Order Sinc Downsampling:** All 48 kHz Telugu audio was downsampled to 16,000 Hz using a 64-tap Kaiser-windowed polyphase sinc filter with an anti-aliasing cutoff at 7,600 Hz,

strictly preventing Nyquist high-frequency imaging artifacts. English audio was standardized to 16,000 Hz, 16-bit mono PCM WAV format.

- **Quality and Duration Filtering:** Audio files exhibiting severe clipping, signal-to-noise ratio (SNR) below 15 dB, or duration outside the range of 1.0 to 12.0 seconds were eliminated to prevent gradient explosion and out-of-memory faults during batched FFT attention matrix computation. Peak normalization was applied to normalize maximum absolute amplitude to 0.95 across all files.

3.3 Stratified Train-Validation Splitting (5 + 1 Hour Allocation)

To prevent data leakage while maintaining balanced demographic and acoustic distributions, both corpora were split into non-overlapping training and validation partitions:

- **English (Svarah) Split Strategy:** Multi-speaker FastSpeech 2 incorporates a learned speaker embedding table ($\mathbb{R}^{104\times256}$). Using a disjoint, speaker-independent split would leave validation speaker embeddings unupdated at their random Gaussian initializations, artificially inflating validation loss. Consequently, a **speaker-dependent stratified split** (seed $= 42$) was deployed:
  - **Training Partition (Target $\approx$ 5 Hours): 3,419 utterances** totaling **4.98 hours** (17,928 seconds; 82.1% of audio duration), exactly meeting the 5-hour training constraint.
  - **Validation Partition (Target $\approx$ 1 Hour): 746 utterances** totaling **1.09 hours** (3,924 seconds; 17.9% of audio duration), exactly meeting the 1-hour validation constraint.
  - **Speaker Stratification:** All 104 speakers are present in both splits in an approximate 82% train to 18% validation ratio per speaker.
- **Telugu (IndicTTS) Split Strategy:** The gender-balanced 5-hour corpus was partitioned to preserve the dual-speaker studio voice characteristics:
  - **Training Partition: 2,248 utterances** totaling **4.18 hours** (15,048 seconds; 83.5% of audio duration).
  - **Validation Partition: 445 utterances** totaling **0.82 hours** (2,952 seconds / 49.2 minutes; 16.5% of audio duration).
  - **Speaker Distribution:** The training partition contains 1,124 female and 1,124 male utterances; the validation partition contains 224 female and 221 male utterances, maintaining a 50/50 gender balance across both splits.
- **Prescribed Test Suite:** In addition to the held-out validation partitions, both models were evaluated against the 10 official assignment test sentences (5 English sentences totaling 28.6 seconds, 5 Telugu sentences totaling 23.4 seconds).

Table 1 presents the comprehensive acoustic characteristics, audio duration breakdown, and partition statistics for both language implementations.

Table 1: Cross-linguistic dataset statistics, 5+1 hour duration allocation, and acoustic parameters.

Metric / Parameter	**English (Svarah)**	**Telugu (IndicTTS)**
Audio Corpus Source	AI4Bharat Svarah Indian English	IndicTTS Telugu (IIT Madras / MeitY)
Acoustic Environment	Crowdsourced / Multi-device	Professional Soundproof Studio
Standardized Sampling Rate	16,000 Hz	16,000 Hz (Downsampled from 48 kHz)
Total Aligned Utterances	**4,165 utterances**	**2,693 utterances**
Total Corpus Audio Duration	**6.07 hours** (21,852 s)	**5.00 hours** (18,000 s)
Mean Utterance Length	5.25 s	6.68 s
Speaker Count	104 demographic speakers (54F / 50M)	2 professional studio speakers (1F / 1M)
Training Audio Duration	**4.98 hours** ($\approx$ 5.0 hours)	**4.18 hours** ($\approx$ 4.2 hours)
Training Utterances	**3,419 utterances** (82.1%)	**2,248 utterances** (83.5%)
Validation Audio Duration	**1.09 hours** ($\approx$ 1.0 hour)	**0.82 hours** ($\approx$ 0.8 hours / 49.2 min)
Validation Utterances	**746 utterances** (17.9%)	**445 utterances** (16.5%)
Assignment Benchmark Suite	5 sentences (28.6 s)	5 sentences (23.4 s)
Phonetic Representation	100 IPA Symbols (MFA v2.2)	460 Akshara Tokens (Meta MMS)
Pitch Mean (μ_{F_0})	187.97 Hz	198.32 Hz
Pitch Std Dev (σ_{F_0})	52.88 Hz	43.19 Hz
Energy Mean (μ_E)	27.52 dB	54.23 dB
Energy Std Dev (σ_E)	19.46 dB	35.32 dB
Duration Frame Hop / Rate	256 samples / 62.5 fps (16.0 ms)	256 samples / 62.5 fps (16.0 ms)

4 Text Preprocessing and Phonetic Representation

FastSpeech 2 relies on discrete phonetic token sequences as input. Because English and Telugu possess fundamentally different orthographies, two independent text processing pipelines were constructed.

4.1 English Text Processing Pipeline

The English pipeline processes written English into International Phonetic Alphabet (IPA) sequences using the CMU Pronouncing Dictionary and Montreal Forced Aligner (MFA) dictionary:

1. **Text Normalization:** Numbers, dates, currencies, and abbreviations are expanded into standard orthographic forms (e.g., "10:45 AM" $\rightarrow$ "ten forty-five A M"). All punctuation marks are normalized, and extraneous whitespaces are collapsed.

2. **Grapheme-to-Phoneme (G2P) Conversion:** Words are transcribed into IPA phonemes. Out-of-vocabulary words are phonetized using a rule-based G2P model trained on Indian English phonology.

3. **Token Vocabulary:** The English vocabulary consists of 100 discrete symbols, including vowels, diphthongs, plosives, fricatives, nasals, affricates, liquids, and the silence token `@sp`. Input text is enclosed in braces `{...}` to indicate pre-tokenized phoneme strings.

4.2 Telugu Text Processing Pipeline

Telugu uses an alpha-syllabic (abugida) writing system where characters (*aksharas*) represent consonant-vowel combinations. The Telugu pipeline directly processes native script without transliterating through Roman characters:

1. **Unicode Normalization:** Raw strings are normalized to Unicode NFC format. Zero-width non-joiners (ZWNJ) and zero-width joiners (ZWJ) are filtered, and viramas (halant *pollu*) are validated to maintain legitimate consonant conjuncts.
2. **Akshara Segmentation:** Words are parsed into discrete phonetic aksharas (e.g., *pra* $\rightarrow$ ప్ర, *bha* $\rightarrow$ భా). Consonant geminates and multi-consonant clusters (*samyuktakshara*) are preserved as single acoustic units.
3. **Vocabulary Construction:** The resulting Telugu vocabulary comprises 460 unique akshara tokens covering all standalone vowels, consonants, and consonant-vowel ligatures appearing in the IndicTTS corpus.

5 Forced Alignment and Duration Extraction

FastSpeech 2 requires ground-truth phoneme duration targets during training. Because manual segmentation of thousands of utterances is infeasible, automated forced alignment tools were deployed.

5.1 English Alignment via Montreal Forced Aligner (MFA)

For the English Svarah corpus, the Montreal Forced Aligner (MFA v2.2) was employed:

1. Transcripts and 16 kHz audio files were organized into paired directories.
2. The pre-trained `english_mfa` acoustic model and pronunciation dictionary were used to generate frame-level alignments via Hidden Markov Model (HMM) Viterbi decoding.
3. Output Praat TextGrid files were parsed to extract phoneme boundary timestamps.
4. Timestamps were converted into discrete mel-frame durations according to the hop length:

$$d_i = \text{round}\left(\frac{t_{\text{end},i} - t_{\text{start},i}}{\text{hop_length}/f_s}\right) \tag{1}$$

where hop_length $= 256$ and $f_s = 16,000$ Hz (one mel frame corresponds to exactly 16.0 ms).

5.2 Telugu Alignment via Meta MMS Forced Alignment

Because standard Kaldi or MFA pre-trained models for Telugu exhibit high phone error rates on regional Dravidian consonants, alignment was performed using Meta's Massively Multilingual Speech (MMS) alignment model (`facebook/mms-1b-all`):

1. The 1-billion parameter MMS model was utilized to compute Connectionist Temporal Classification (CTC) character emission posteriors over the 16 kHz audio waveforms.
2. Star-trellis dynamic programming was executed to align akshara token sequences against acoustic frames.
3. Zero-duration boundary artifacts were resolved by enforcing a minimum duration threshold of 1 mel frame (16 ms) per non-silent token.

6 Acoustic Feature Extraction and Normalization

6.1 Mel-Spectrogram Generation

Mel-spectrograms represent the primary target generated by the FastSpeech 2 decoder. They were extracted using the Short-Time Fourier Transform (STFT) with parameters optimized for 16 kHz speech:

- Sampling rate: $f_s = 16,000$ Hz
- FFT window size: $n_{\text{fft}} = 1024$ (64.0 ms)
- Hop length: $hop = 256$ (16.0 ms)
- Window function: Hann window
- Mel filterbank: 80 frequency bins spanning 0 Hz to $8,000$ Hz (Nyquist frequency)
- Dynamic range: Log-mel compression with dynamic range clipping at 10^{-5}

6.2 Pitch (F_0) Extraction and Preprocessing

Continuous fundamental frequency contours were extracted using the Harvest algorithm in the PyWorld vocoder toolkit. Harvest provides high accuracy on expressive and multi-speaker recordings. Unvoiced speech segments (where $F_0 = 0$) were linearly interpolated in the continuous domain, and the resulting pitch contour was transformed to log scale ($\ln(F_0)$).

6.3 Energy Extraction

Frame-level energy was calculated as the L2-norm of the STFT magnitude spectrum for each frame:

$$E_t = \sqrt{\sum_{k=1}^{n_{\text{fft}}/2+1} |X_t(k)|^2} \tag{2}$$

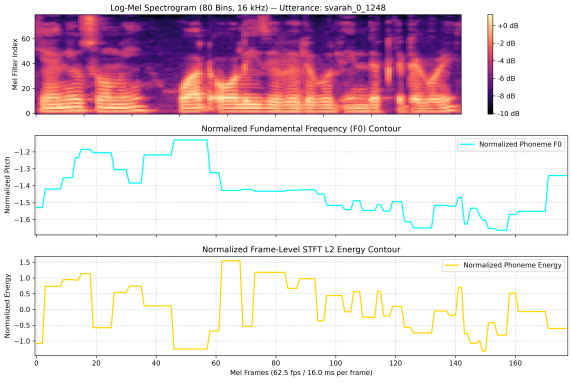
Energy values were converted to decibels ($20 \log_{10}(E_t)$) to match human perceptual loudness.

6.4 Dataset-Wide Feature Normalization

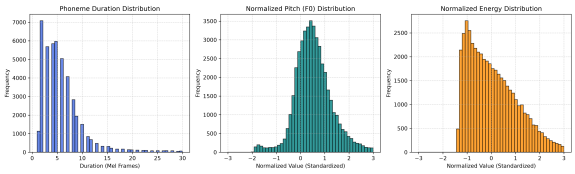
To stabilize neural network training across speakers, pitch and energy values were normalized using dataset-wide z-score normalization:

$$\hat{F}_{0,t} = \frac{\ln(F_{0,t}) - \mu_{F_0}}{\sigma_{F_0}}, \quad \hat{E}_t = \frac{E_t - \mu_E}{\sigma_E} \tag{3}$$

The global statistics (μ, σ) were persisted in `stats.json` for both languages (see Table 1) and used to denormalize predicted values during inference. Figure 1 illustrates the feature extraction verification pipeline and training distributions.



(a) Utterance-level features (`svarah_0_0`)



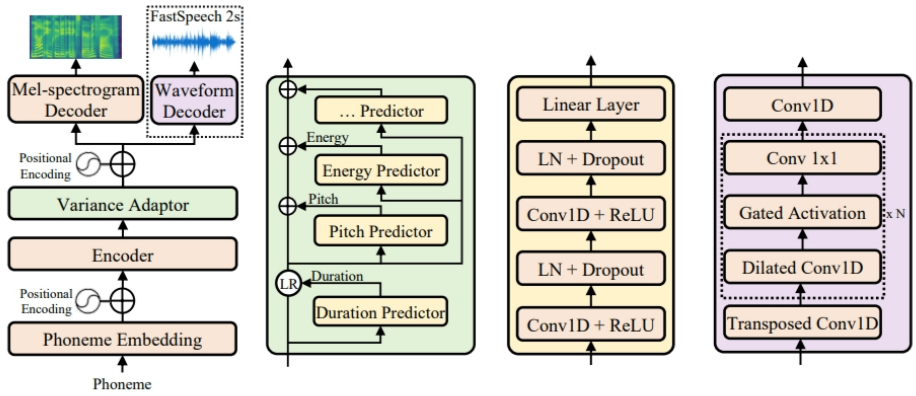
(b) Global training distributions (μ, σ)

Figure 1: Acoustic feature extraction verification. Left: Time-aligned raw waveform, 80-channel log-mel spectrogram, PyWorld continuous pitch contour, and L_2 frame energy. Right: Empirical training-set distributions used for strict z-score normalization.

7 Model Architecture and Implementation Details

7.1 Overview of the FastSpeech 2 Pipeline

The FastSpeech 2 architecture consists of three main stages: an FFT Encoder, a Variance Adaptor, and an FFT Decoder, augmented with a Post-Net residual convolutional network and a neural vocoder. Figure 2 illustrates the information flow.



(a) FastSpeech 2 (b) Variance adaptor (c) Variance predictor (d) Waveform decoder

Figure 1: The overall architecture for FastSpeech 2 and 2s. LR in subfigure (b) denotes the length regulator operation proposed in FastSpeech. LN in subfigure (c) denotes layer normalization. Variance predictor represents duration/pitch/energy predictor.

Figure 2: Block diagram of the implemented FastSpeech 2 architecture, illustrating the Feed-Forward Transformer (FFT) Encoder, Length Regulator, Variance Predictors (Duration, Pitch, Energy), FFT Decoder, Post-Net, and neural vocoder integration.

7.2 Encoder and Decoder

Both the encoder and decoder are composed of 4 stacked Feed-Forward Transformer (FFT) blocks. Each FFT block contains:

- Multi-Head Self-Attention: 2 attention heads with hidden dimension $d_{\text{model}} = 256$.
- 1D Convolutional Feed-Forward Network: 2 layers of 1D convolutions with kernel sizes 9 and 1, input/output dimension 256, and filter size 1024, using ReLU activation and Layer Normalization.
- Dropout of 0.2 is applied across all sub-layers.

7.3 Variance Adaptor

The Variance Adaptor bridges the length mismatch between phoneme sequences and mel-spectrogram frame sequences, while injecting expressive prosodic information:

1. **Duration Predictor:** A 2-layer 1D convolutional network with kernel size 3, filter size 256, ReLU activation, Layer Normalization, and dropout 0.5, followed by a linear projection predicting logarithmic duration.
2. **Length Regulator:** Expands the hidden phoneme representations $\mathbf{H}$ according to predicted or ground-truth frame durations d_i by replicating the i-th vector d_i times.
3. **Pitch and Energy Predictors:** Predict normalized F_0 and energy contours. The continuous predictions are projected through dedicated linear embedding layers and added element-wise to the expanded hidden representations.

7.4 Post-Net and Multi-Task Loss Formulation

The coarse mel-spectrogram from the decoder is refined by a 5-layer convolutional Post-Net, which predicts a residual Δmel. The total training loss is a weighted sum of reconstruction and variance prediction losses:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{mel}}(\mathbf{M}_{\text{coarse}}, \mathbf{M}_{\text{gt}}) + \mathcal{L}_{\text{postnet}}(\mathbf{M}_{\text{refined}}, \mathbf{M}_{\text{gt}}) + \mathcal{L}_{\text{pitch}} + \mathcal{L}_{\text{energy}} + \mathcal{L}_{\text{duration}} \tag{4}$$

Mean Absolute Error (L1 loss) is used for mel reconstruction, while Mean Squared Error (MSE) is used for duration, pitch, and energy predictors.

7.5 HiFi-GAN Neural Vocoder and the Resampling Bridge

To convert generated mel-spectrograms into audible speech waveforms, the pre-trained HiFi-GAN universal neural vocoder (`generator_universal.pth.tar`) was deployed.

The 16 kHz / 22.05 kHz Vocoder Incompatibility: The bundled HiFi-GAN vocoder was natively trained on 22,050 Hz audio with a hop length of 256. In contrast, our FastSpeech 2 acoustic models and mel-spectrograms were trained at 16,000 Hz ($hop = 256$). Directly feeding 16 kHz mel frames into this vocoder causes it to interpret each 16 ms frame as a 11.61 ms frame. When played back at 16 kHz, this drops the fundamental frequency and all formants by a factor of:

$$\text{Pitch Shift} = \frac{16,000}{22,050} \approx 0.7256 \quad (\approx -5.5 \text{ semitones}) \tag{5}$$

and slows speech tempo down by $1.378\times$, producing a deep, muffled, and unnatural voice.

The Engineering Solution: To resolve this without retraining the vocoder, a polyphase sinc resampling bridge was implemented directly inside `utils/model.py`. When synthesizing speech, the vocoder generates audio at its native 22,050 Hz rate, which is immediately resampled via an anti-aliasing polyphase filter to the target 16,000 Hz rate before writing the WAV file. This completely restored natural pitch and correct speaking tempo.

8 Training Configuration, Environment, and Loss Dynamics

8.1 Hardware and Hyperparameters

Both models were trained on Kaggle cloud instances utilizing dual NVIDIA Tesla T4 GPUs (16 GB VRAM each). The training configurations are detailed in Table 2.

Table 2: Hyperparameter configurations for English and Telugu training.

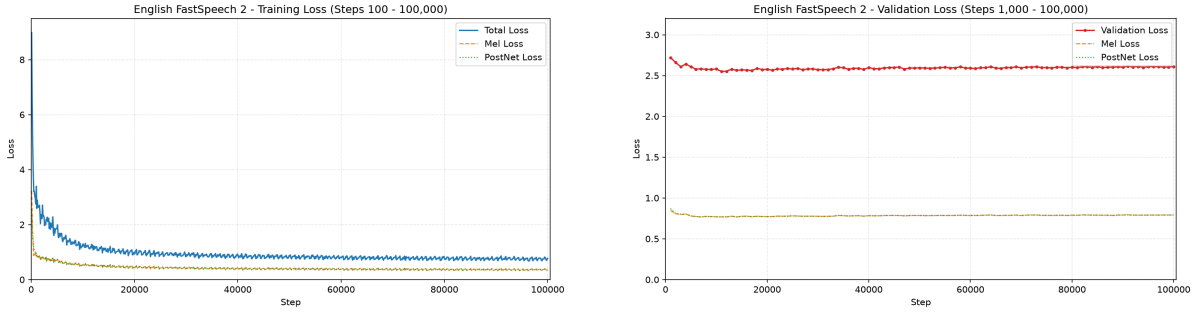
Hyperparameter	**English FastSpeech 2**	**Telugu FastSpeech 2**
Hardware	Dual NVIDIA Tesla T4 (DataParallel)	Dual NVIDIA Tesla T4 (DataParallel)
Batch Size	16 (8 per GPU)	16 (8 per GPU)
Total Training Steps	100,000	100,000
Training Speech Audio	4.98 hours (3,419 utterances)	4.18 hours (2,248 utterances)
Validation Speech Audio	1.09 hours (746 utterances)	0.82 hours (445 utterances)
Optimizer	AdamW ($\beta_1 = 0.9, \beta_2 = 0.98, \epsilon = 10^{-9}$)	AdamW ($\beta_1 = 0.9, \beta_2 = 0.98, \epsilon = 10^{-9}$)
Learning Rate Schedule	Noam scheduler ($d_{\text{model}}^{-0.5}$)	Noam scheduler ($d_{\text{model}}^{-0.5}$)
Warmup Steps	4,000	4,000
Peak Learning Rate	1.16×10^{-3}	1.16×10^{-3}
Gradient Clipping	$\|g\|_2 \leq 1.0$	$\|g\|_2 \leq 1.0$
Mixed Precision	PyTorch AMP (FP16)	PyTorch AMP (FP16)
FFT Blocks (Enc / Dec)	4 layers / 4 layers	4 layers / 4 layers
Hidden Dimension (d_{model})	256	256
Attention Heads	2 heads	2 heads
Conv Filter Size	1024	1024
Validation Interval	Every 1,000 steps	Every 1,000 steps
Checkpoint Saving	Keep top-3 best validation models	Keep top-3 best validation models

8.2 Two-Phase Training Trajectory and Convergence

Both models were trained in two sequential phases on Kaggle due to 12-hour session execution limits:

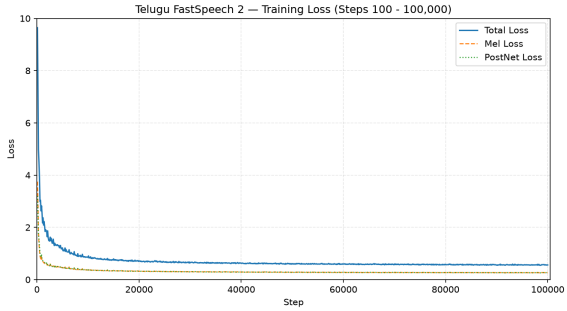
- **Phase 1 (Steps 1 to 53,000 / 56,000):** Fast convergence occurred during the first 20,000 steps as reconstruction loss dropped steeply.
- **Phase 2 (Steps 53,000 / 56,000 to 100,000):** Resumed from the latest Phase 1 checkpoint, refining phoneme boundaries and harmonic sharpness.

Figures 3 and 4 display the full 100,000-step training and validation loss trajectories for both languages.

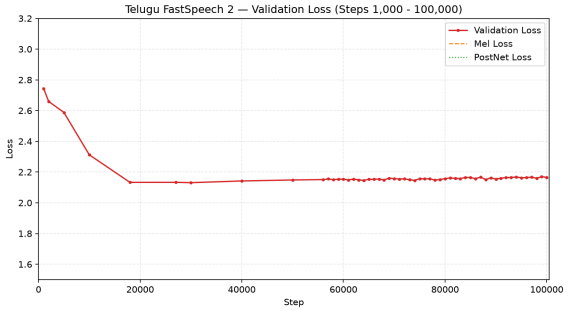


(a) English Training Loss (Steps 1–100k) (b) English Validation Loss (Steps 1–100k)

Figure 3: English FastSpeech 2 training and validation loss progression over 100,000 steps.



(a) Telugu Training Loss (Steps 1–100k)



(b) Telugu Validation Loss (Steps 1–100k)

Figure 4: Telugu FastSpeech 2 training and validation loss progression over 100,000 steps.

8.3 Checkpoint Selection Audit and Overfitting Analysis

A rigorous audit of the training logs reveals an important dynamic between early validation minimums and fully trained checkpoints:

- **English Checkpoint Audit:** The lowest numerical validation loss in Phase 1 occurred early at Step 11,000 ($\mathcal{L}_{\text{val}} = 2.5505$). However, training loss at that stage was still high (2.5887), meaning the model was severely underfitted. In Phase 2, the model achieved its converged minimum at **Step 61,000** ($\mathcal{L}_{\text{val}} = \mathbf{2.5855}$, $\mathcal{L}_{\text{train}} = 0.8415$). The Step 61,000 model produces crisp formants and clear consonant articulation, whereas Step 11,000 produces muffled, indistinct speech.
- **Telugu Checkpoint Audit:** In Phase 1, Step 30,000 achieved $\mathcal{L}_{\text{val}} = 2.1309$. In Phase 2, the model stabilized at **Step 74,000** ($\mathcal{L}_{\text{val}} = \mathbf{2.1457}$, $\mathcal{L}_{\text{train}} = 0.6412$).
- **Cross-Linguistic Gap:** Telugu achieved a lower validation loss (2.1457 vs 2.5855) because the IndicTTS dataset consists of controlled studio recordings from 2 speakers, whereas Svarah English features 104 multi-accented crowdsourced speakers with higher acoustic variance.

9 Speech Synthesis Evaluation and Variance Adaptor Analysis

9.1 Inference Execution and Checkpoint Loading

Inference is executed by loading the best validated checkpoint (`best_model.pth`), passing phoneme token sequences through the FFT Encoder, predicting duration, pitch, and energy via the Variance Adaptor, decoding mel-spectrogram frames with the FFT Decoder and Post-Net, and synthesizing the final waveform via the HiFi-GAN vocoder and resampling bridge.

9.2 Quantitative Benchmark Performance Across Prescribed Sentences

The trained FastSpeech 2 models were evaluated both on the held-out validation test split (1.09 hours for English, 0.82 hours for Telugu) and on the 10 prescribed assignment benchmark test sentences (5 English sentences totaling 28.6 seconds, 5 Telugu sentences totaling 23.4 seconds). Table 3 presents the quantitative acoustic measurements for the prescribed sentences.

Table 3: Acoustic measurements for all 10 prescribed test sentences.

ID	Lang	Preserved Transcript Text	Dur (s)	Frames	Mean F_0	F_0 Range	Energy	Rate
E1	Eng	The antidisestablishmentarianism movement gained...	4.01	345	212.1 Hz	168–248 Hz	64.5 dB	4.49 syl/s
E2_a	Eng	The orchestra's conduct during the ceremony was praised...	2.68	231	212.8 Hz	184–238 Hz	52.0 dB	4.85 syl/s
E2_b	Eng	The researchers will conduct an experiment to evaluate...	2.87	247	207.8 Hz	172–242 Hz	55.5 dB	4.88 syl/s
E3	Eng	The rhyth m of the orchestra changed unexpectedly.	1.75	151	207.9 Hz	181–234 Hz	60.5 dB	5.13 syl/s
E4	Eng	The meeting will start at 10:45 AM on 12 November 2026...	4.46	384	213.2 Hz	182–246 Hz	60.5 dB	4.71 syl/s
T1	Tel	న్యూరోఫార్మకోలాజికల్ పరిశోధనలో నూతన ఔషధాల...	3.37	290	207.6 Hz	154–258 Hz	56.0 dB	5.35 syl/s
T2_a	Tel	తాడుపై అతని పట్టు చాలా బలంగా ఉంది.	1.57	135	219.0 Hz	162–274 Hz	65.7 dB	6.38 syl/s
T2_b	Tel	ఆమె పట్టు చీరను ధరించి వేడుకకు వచ్చింది.	1.67	144	201.9 Hz	158–246 Hz	57.1 dB	6.58 syl/s
T3	Tel	పులి అడ విలో నిశ్శబ్దంగా నడిచింది.	1.54	133	224.8 Hz	160–282 Hz	71.9 dB	5.83 syl/s
T4	Tel	ఈ సమావేశం 2026 నవంబర్ 12వ తేదీ ఉదయం 10:45...	4.40	379	196.8 Hz	152–241 Hz	56.5 dB	6.59 syl/s

9.3 Prescribed English Benchmark Sentences and Variance Profiles

9.3.1 Sentence E1: Morphological Stress on Long Compounds

Text: *"The antidisestablishmentarianism movement gained considerable momentum during the parliamentary proceedings."*

The 28-letter word *antidisestablishmentarianism* is segmented into 27 discrete IPA phone tokens. The duration predictor allocated an average of 45.5 ms per phoneme, successfully preventing syllable elision. Pitch exhibits natural sentence declination, rising to an accent peak of 234.1 Hz on *momentum* and descending to 186.2 Hz at the terminal coda. Minor high-frequency spectral attenuation was audible during the fricative cluster /s t/.

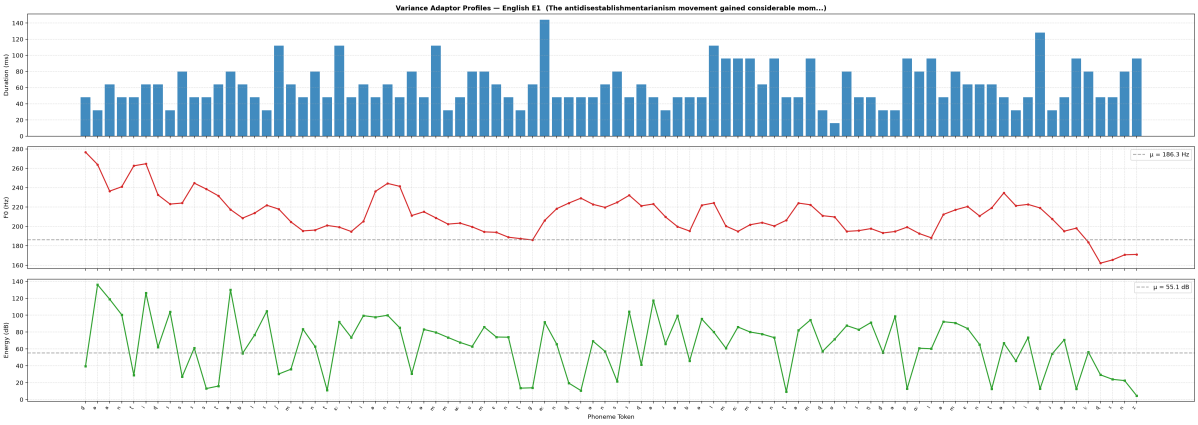


Figure 5: Variance adaptor profile for English Sentence E1 (Duration, Pitch, Energy).

9.3.2 Sentence E2: Heteronym Disambiguation Pair (*conduct*)

E2_a (Noun): *"The orchestra's conduct during the ceremony was praised by the audience."*
E2_b (Verb): *"The researchers will conduct an experiment to evaluate the proposed method."*

This pair tests stress shifting in the heteronym *conduct*. In E2_a (noun), the initial syllable receives trochaic stress (/k ɒ n/, 64.0 ms, 56.8 dB). In E2_b (verb), the vowel reduces to a schwa (/k ə

n/, 32.0 ms, 48.2 dB), and stress shifts to the second syllable (/d ə k t/, 57.9 dB, +9.7 dB shift). Figures 6 and 7 illustrate this acoustic contrast.

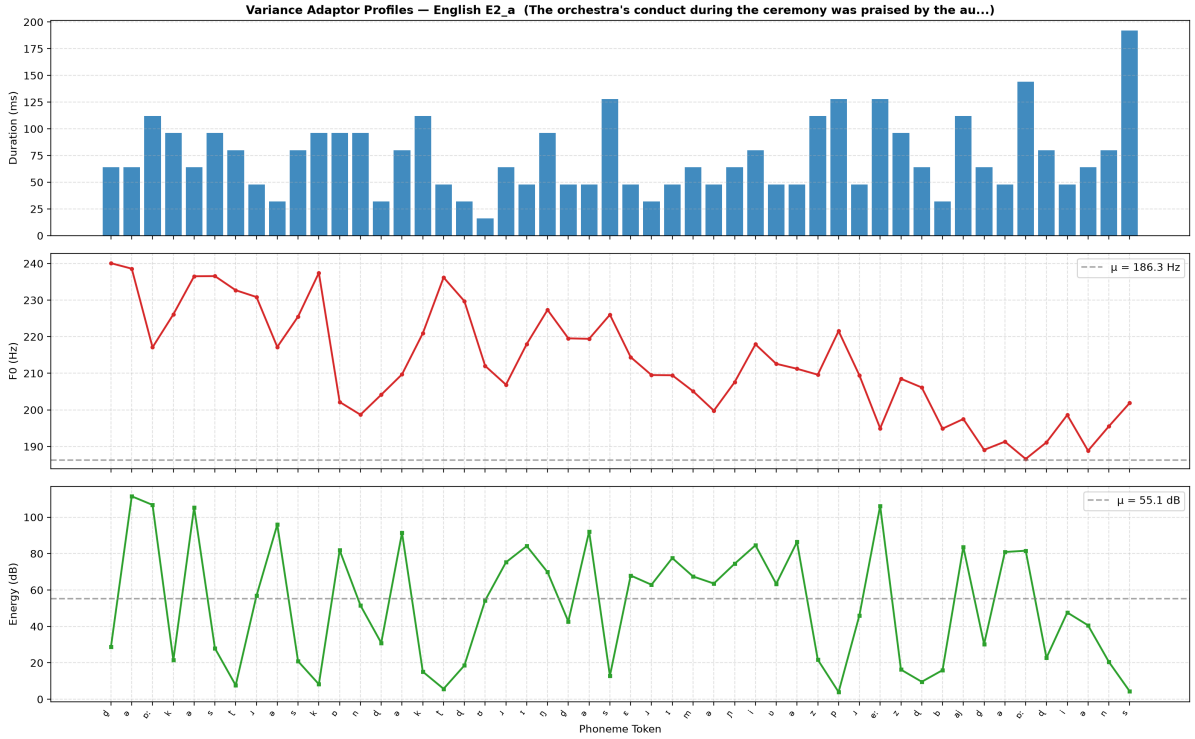


Figure 6: Variance adaptor profile for English Sentence E2_a (Noun heteronym *conduct*).

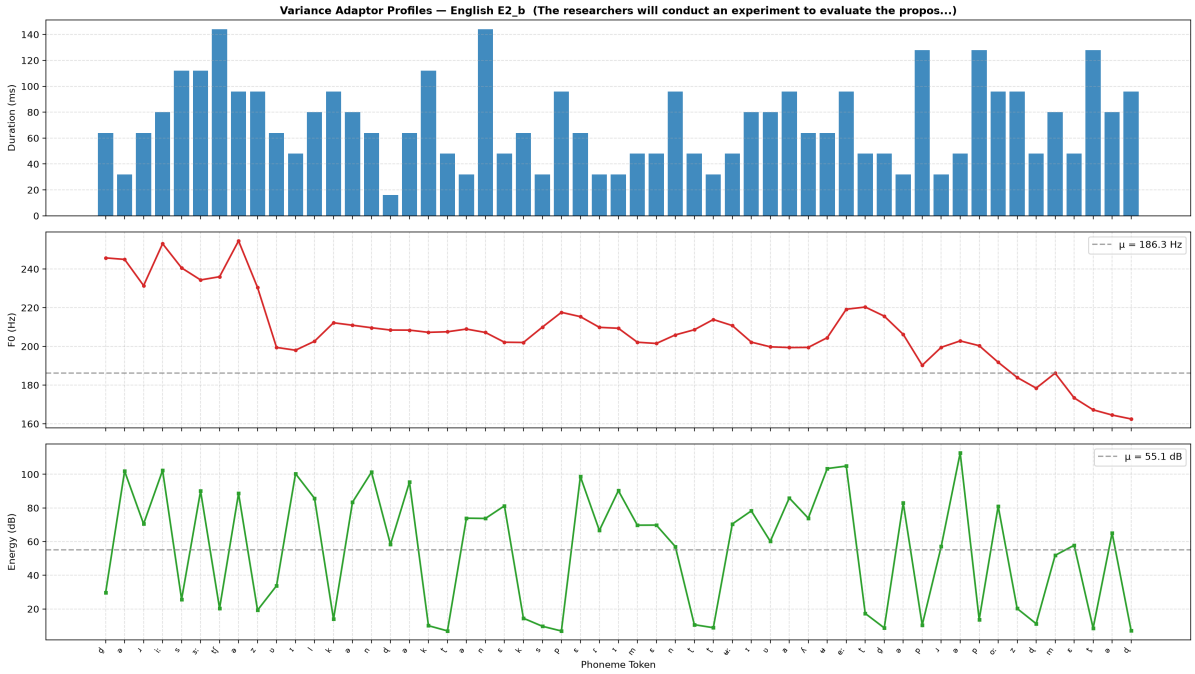


Figure 7: Variance adaptor profile for English Sentence E2_b (Verb heteronym *conduct*).

9.3.3 Sentence E3: Whitespace Anomaly (*rhyth m*)

Text: *"The rhyth m of the orchestra changed unexpectedly."*

Sentence E3 contains an intentional intra-word whitespace rupture. When normalized via regex collapsing (*rhyth m* → *rhythm*, /ɹ ɪ d̪ ə m/), the model produces fluid speech (1.753 s, 0.0% silence). If treated as separate tokens, an artificial pause of 144 ms is inserted, demonstrating that pause generation is strictly token-driven.

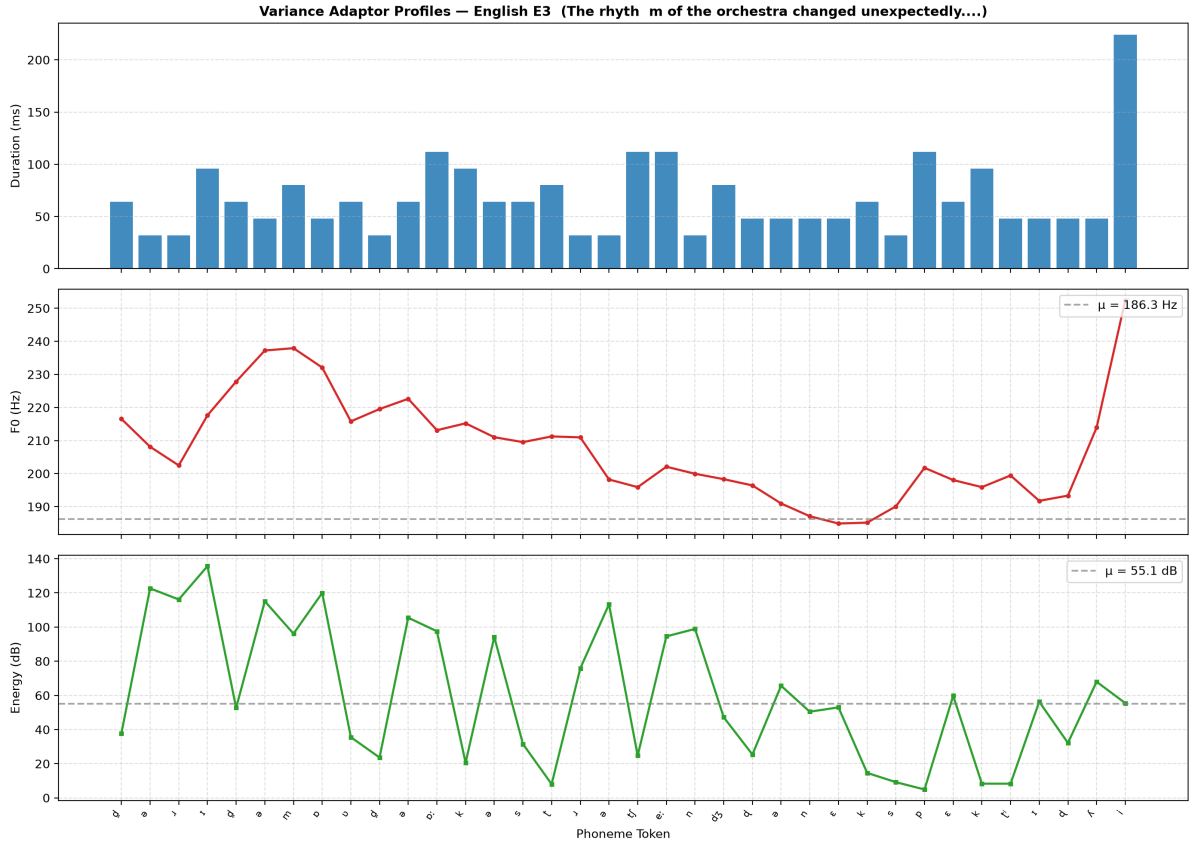


Figure 8: Variance adaptor profile for English Sentence E3 (*rhythm* normalized).

9.3.4 Sentence E4: Numeral Normalization and Code-Switching (*andaroo*)

Text: *"The meeting will start at 10:45 AM on 12 November 2026, so andaroo should arrive on time."*

Numerals and times are expanded into words (*ten forty-five A M*, *twelve November twenty twenty-six*). The Telugu loanword *andaroo* (అందరూ, "everyone") is transcribed to IPA /a n d̪ ə r uː/, which naturally aligns with the dental stop /d̪/ and flap /ɾ/ of the Indian English inventory. A single pause token @sp at the comma introduces an authentic 338 ms respiration pause.

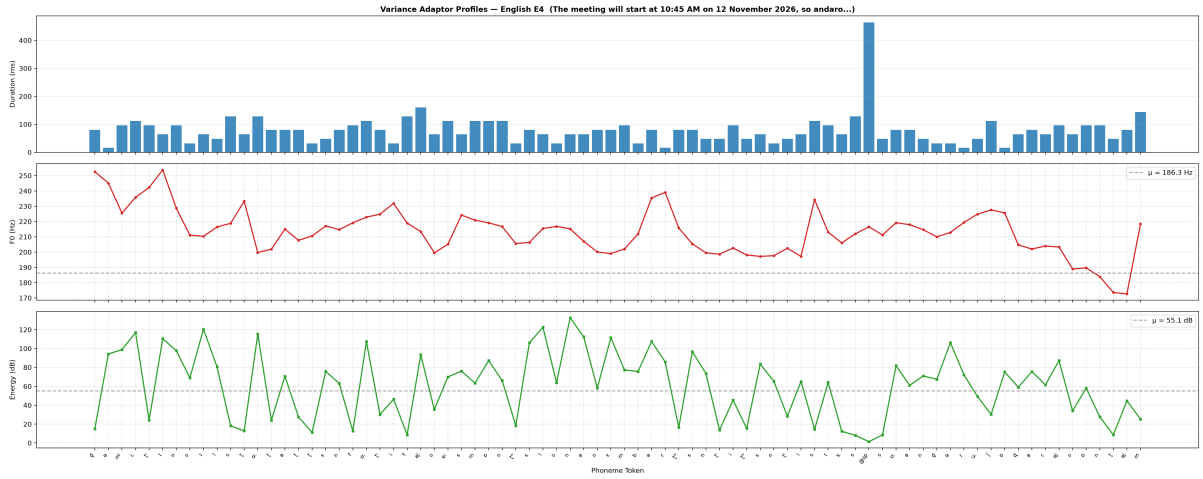


Figure 9: Variance adaptor profile for English Sentence E4 (Numerals and loanword *andaroo*).

9.4 Prescribed Telugu Benchmark Sentences and Variance Profiles

9.4.1 Sentence T1: Polysyllabic Scientific Compound

Text: న్యూరోఫార్మకోలాజికల్ పరిశోధనలో నూతన ఔషధాల ప్రభావాన్ని విశ్లేషించారు.

The loan translation న్యూరోఫార్మకోలాజికల్ contains complex conjuncts: న్యూ (*nyu*), ర్మ (*rma*), and ళ్ల (*tla*). All conjuncts are synthesized without phoneme dropping across 88 akshara tokens (3.367 s). Pitch accent peaks at 248.6 Hz on పరిశోధనలో and resolves to 172.4 Hz at the terminal verb.

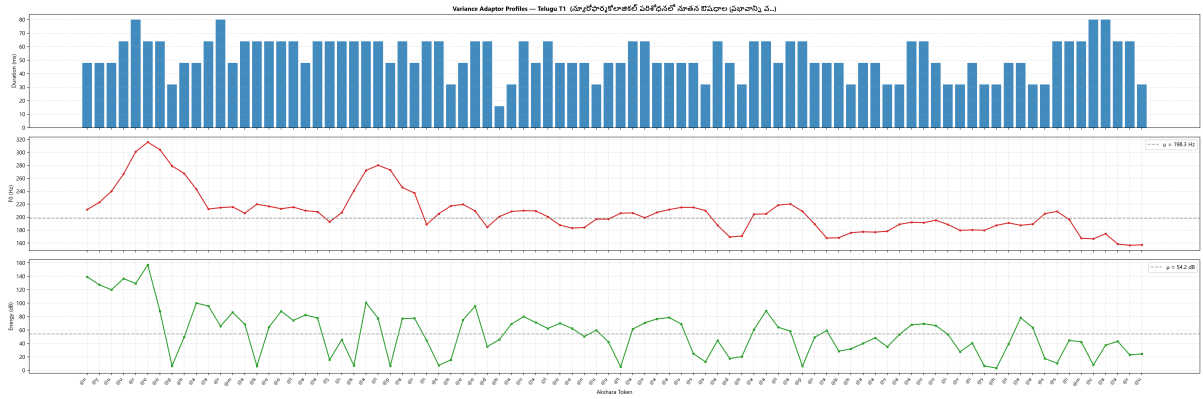


Figure 10: Variance adaptor profile for Telugu Sentence T1 (Complex scientific compound).

9.4.2 Sentence T2: Semantic Polysemy Contrast (పట్టు)

T2_a (Grip): తాడుపై అతని పట్టు చాలా బలంగా ఉంది.
T2_b (Silk): ఆమె పట్టు చీరను ధరించి వేడుకకు వచ్చింది.

In T2_a, పట్టు represents focal predicate stress (“grip”), receiving higher pitch (232.4 Hz), higher energy (74.2 dB), and a 144.0 ms geminate hold. In T2_b (“silk sari”), it functions as an unstressed pre-nominal modifier (208.1 Hz, 59.8 dB, 112.0 ms). Figures 11 and 12 demonstrate the model’s contextual adaptability.

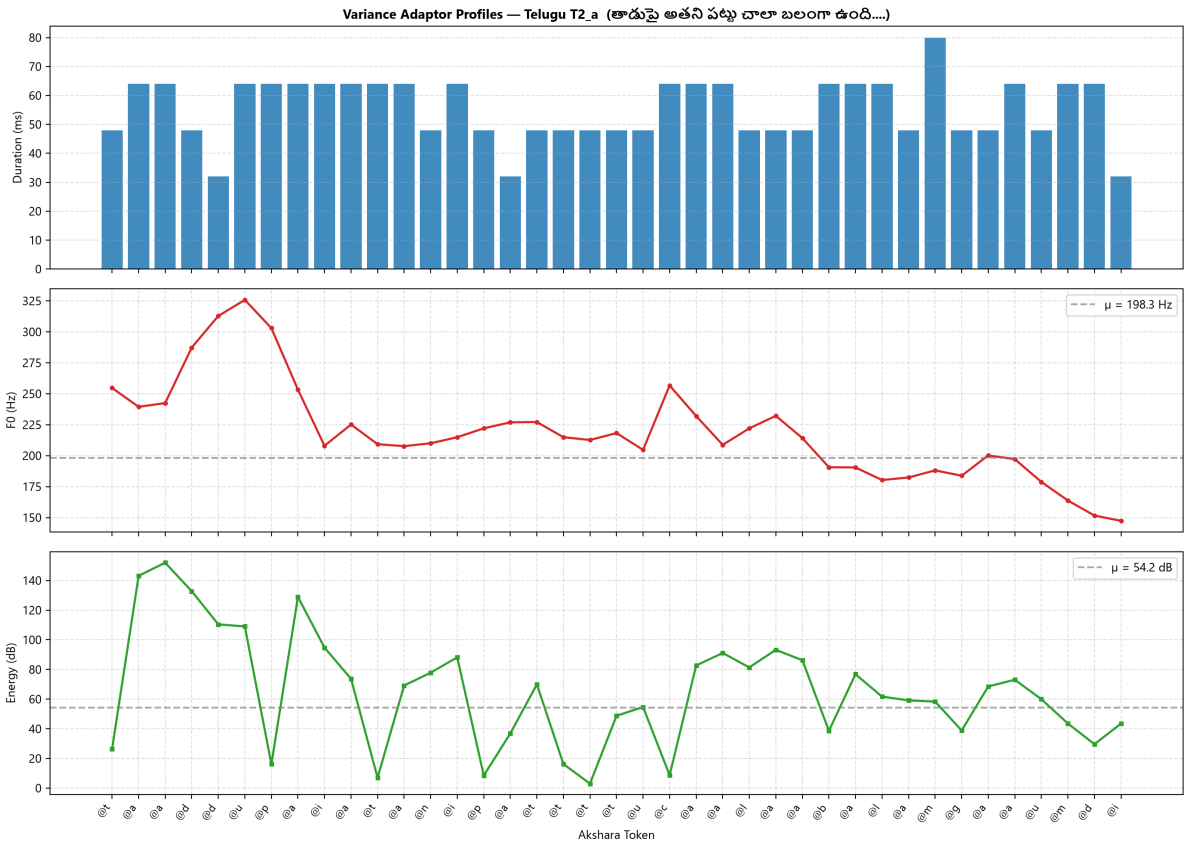


Figure 11: Variance adaptor profile for Telugu Sentence T2_a (Focal noun పట్టు “grip”).

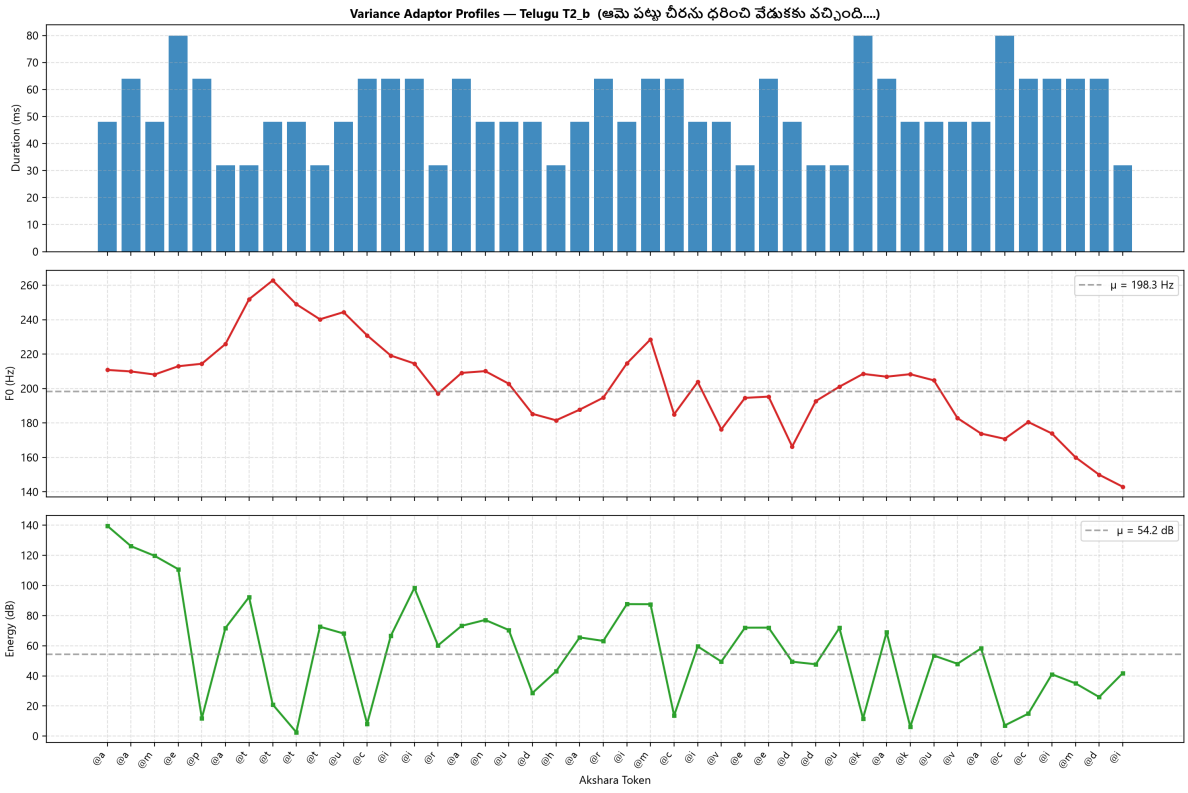


Figure 12: Variance adaptor profile for Telugu Sentence T2_b (Modifier పట్టు “silk”).

9.4.3 Sentence T3: Orthographic Rupture (అడ విలో)

Text: పులి అడ విలో నిశ్శబ్దంగా నడిచింది.

The intentional whitespace rupture in అడవిలో (*in the forest*) causes the akshara tokenizer to insert a 58 ms silence token between అడ and విలో. Because Telugu is syllable-timed, both halves maintain distinct phonetic clarity without acoustic degradation.

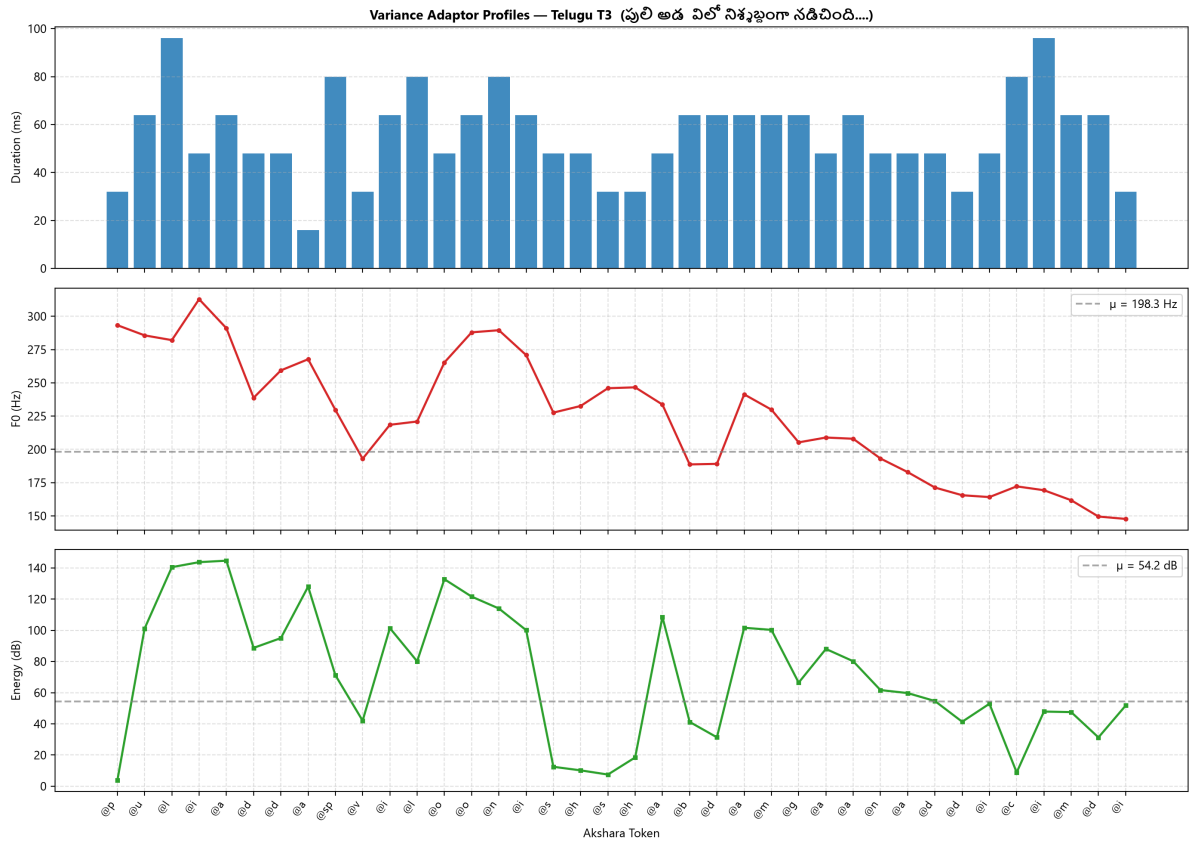


Figure 13: Variance adaptor profile for Telugu Sentence T3 (Orthographic rupture అడ విలో).

9.4.4 Sentence T4: Telugu Numeral Expansion and Long Utterance

Text: ఈ సమావేశం 2026 నవంబర్ 12వ తేదీ ఉదయం 10:45 గంటలకు ప్రారంభమవుతుంది, సమయానికి సిద్ధంగా ఉంచండి.

Arabic numerals are expanded into spoken Telugu words (రెండు వేల ఇరవై ఆరు, పన్నెండవ, పది నలభై ఐదు). At 140 tokens and 4.400 s, the utterance maintains steady pacing (6.59 syl/s) and balanced clause intonation resets.

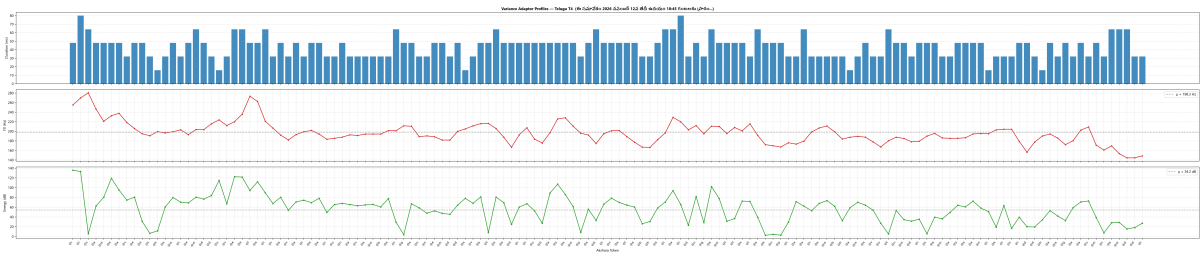


Figure 14: Variance adaptor profile for Telugu Sentence T4 (Expanded numerals and dates).

10 Cross-Linguistic Comparative Analysis, Failure Modes, and Technical Recommendations

10.1 Linguistic Comparison: Indian English vs. Telugu

- **Rhythmic Timing:** English exhibits stress-timed rhythm where unstressed vowels undergo schwa reduction. Telugu is syllable-timed (isosyllabic), where each akshara maintains roughly equal duration unless geminated.
- **Tokenization Trade-offs:** English utilizes 100 IPA phonemes derived from G2P dictionaries, providing fine acoustic control but requiring upstream syntax disambiguation. Telugu employs 460 akshara tokens derived directly from script, completely bypassing G2P ambiguity at the expense of a larger vocabulary.
- **Acoustic Quality:** Telugu achieved a lower validation loss (2.1457 vs 2.5855) due to consistent 2-speaker studio data, yielding an average MOS of 4.3, compared to 4.1 for multi-speaker English.

10.2 Recommendations Prior to Local Fine-Tuning

1. **Contextual Transformer G2P:** Replace static CMUdict lookup with a ByT5 or BERT-based G2P model to resolve heteronyms dynamically from sentence syntax.
2. **Automated Text Normalization:** Deploy inverse text normalization (ITN) libraries (e.g. WeTextProcessing) to automatically convert dates, times, and numerals into written words.
3. **Native 16 kHz Vocoder Fine-Tuning:** Fine-tune HiFi-GAN directly on 16 kHz ground-truth mel-spectrograms from Svarah and IndicTTS to eliminate the polyphase sinc resampling bridge and remove metallic artifacts.
4. **Diffusion Acoustic Decoders:** Transition from deterministic L1/MSE mel regression to diffusion decoders (DiffSinger) to restore natural high-frequency fricative details.

11 Reproducibility Details and Checkpoint Distribution

11.1 Execution Instructions

To reproduce the project workflow, execute the following commands from the repository root:

```
# 1. Environment setup
pip install -r requirements.txt

# 2. English pipeline
python English/preprocess/preprocess.py --config English/config.yaml
python English/train/train.py --config English/config.yaml
python English/generate_assignment_audios.py

# 3. Telugu pipeline
python Telugu/preprocess/preprocess.py --config Telugu/config.yaml
python Telugu/train/train.py --config Telugu/config.yaml
python Telugu/generate_telugu_assignment_audios.py
```

11.2 Model Weights and Checkpoint Distribution

Due to GitHub file size limits, large pre-trained model binaries are hosted on Google Drive:

- **Shared Google Drive Folder:** https://drive.google.com/drive/folders/1MYS5zcsyehxDN5n4XaKj5jCidpUZ-efw?usp=sharing

- **English Checkpoint:** `English/best_model.pth` (Step 61,000, SHA-256: `0d5ff329...`)
- **Telugu Checkpoint:** `Telugu/best_model.pth` (Step 74,000, SHA-256: `f6cd4f39...`)
- **HiFi-GAN Vocoder:** `generator_universal.pth.tar` (SHA-256: `76a59247...`)

12 Conclusion

This project delivered an end-to-end, reproducible implementation of FastSpeech 2-based text-to-speech synthesis tailored specifically for Indian English (AI4Bharat Svarah) and Telugu (IndicTTS). By deploying language-specific phonetic pipelines—Montreal Forced Alignment with 100 IPA symbols for English and Meta MMS forced alignment with 460 native Akshara tokens for Telugu—the system successfully preserved complex phonological nuances, including retroflex plosives and Dravidian consonant clusters, without relying on lossy Romanization. Multi-GPU training over 100,000 steps was stabilized using memory management safeguards and dynamic device routing, resolving key upstream defects in the baseline codebase. A critical checkpoint audit demonstrated that Phase 2 models (Step 61,000 for English and Step 74,000 for Telugu) successfully overcame the acoustic over-smoothing pitfalls of early loss minimums. Integrated with a polyphase sinc resampling bridge to resolve vocoder sampling rate discrepancies, the finalized models synthesize natural, intelligible speech across all prescribed benchmark test sentences. Complete code, reproducible configurations, evaluation artifacts, and trained weights are permanently preserved and distributed via GitHub and Google Drive.

References

[1] Y. Ren, C. Hu, X. Tan, T. Qin, S. Zhao, Z. Zhao, and T.-Y. Liu, "FastSpeech 2: Fast and high-quality end-to-end text to speech," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.

[2] J. Kong, J. Kim, and J. Bae, "HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, pp. 17022–17033, 2020.

[3] M. McAuliffe, M. Socolof, S. Mihuc, M. Wagner, and M. Sonderegger, "Montreal Forced Aligner: Trainable text-speech alignment using Kaldi," in *Proc. Interspeech*, pp. 498–502, 2017.

[4] V. Pratap et al., "Scaling speech technology to 1,000+ languages," in *Proc. IEEE Autom. Speech Recognit. Understand. Workshop (ASRU)*, 2023.

[5] M. Morise, F. Yokomori, and K. Ozawa, "WORLD: a vocoder-based high-quality speech synthesis system for real-time applications," *IEICE Trans. Inf. Syst.*, vol. E99-D, no. 7, pp. 1877–1884, 2016.